Solution: Custom Sort String

Problem Statement

You are given two strings `pattern` and `text`. It is given that all characters of the `pattern` string are unique.

Rearrange the characters of the `text` string based on the characters' order in `pattern` string. In other words, if a character `a` occurs before a character `b` in the `pattern` string, then `a` should occur before `b` in the output string.

Characters in `text` that do not appear in `pattern` should be appended at the end of the rearranged string in their original order.

Examples

1. **Example 1**:
    - **Input**: `pattern = "xy", text = "yyzx"`
    - **Expected Output**: `"xyyz"`
    - **Justification**: In 'pattern', the order is 'x', 'y'. In the `text`, 'x' appears once, 'y' twice, and 'z' doesn't appear. So, we put `z` at the end of the string. Thus, the output string is `xzyy`.
2. **Example 2**:
    - **Input**: `pattern = "abc", text = "aabbcc"`
    - **Expected Output**: `"aabbcc"`
    - **Justification**: Here, the 'text' already follows the order 'a', 'b', 'c' as specified in 'pattern', so the order remains 'aabbcc'.
3. **Example 3**:
    - **Input**: `pattern = "mno", text = "onomon"`
    - **Expected Output**: `"mnnooo"`
    - **Justification**: According to 'pattern', 'm' comes first, then 'n', then 'o'. Rearranging 'text' in this order results in 'm' once, 'n' twice, followed by 'o' thrice, hence 'mnnooo'.

Solution

To solve this problem, we will create a custom sorting mechanism based on the 'pattern' string. The key idea is to prioritize the characters in 'text' that appear in 'pattern', arranging them in the order they appear in 'pattern', and then appending the remaining characters from 'text' in their original order. This approach works because it respects the specified order in 'pattern' while maintaining the sequence of characters in 'text' that are not in 'pattern'.

Our approach involves first counting the frequency of each character of the `text` string. We'll use a hashmap for this purpose. Then, we'll iterate over each character in 'pattern', and if that character exists in `text`, we'll append it to our result string as many times as it appears in `text`. Finally, we'll go through `text` again and append any characters to the result string that were not in `pattern`. This ensures that characters in `text` are first ordered according to `pattern` and then the remaining characters retain their original order.

Step-by-Step Algorithm

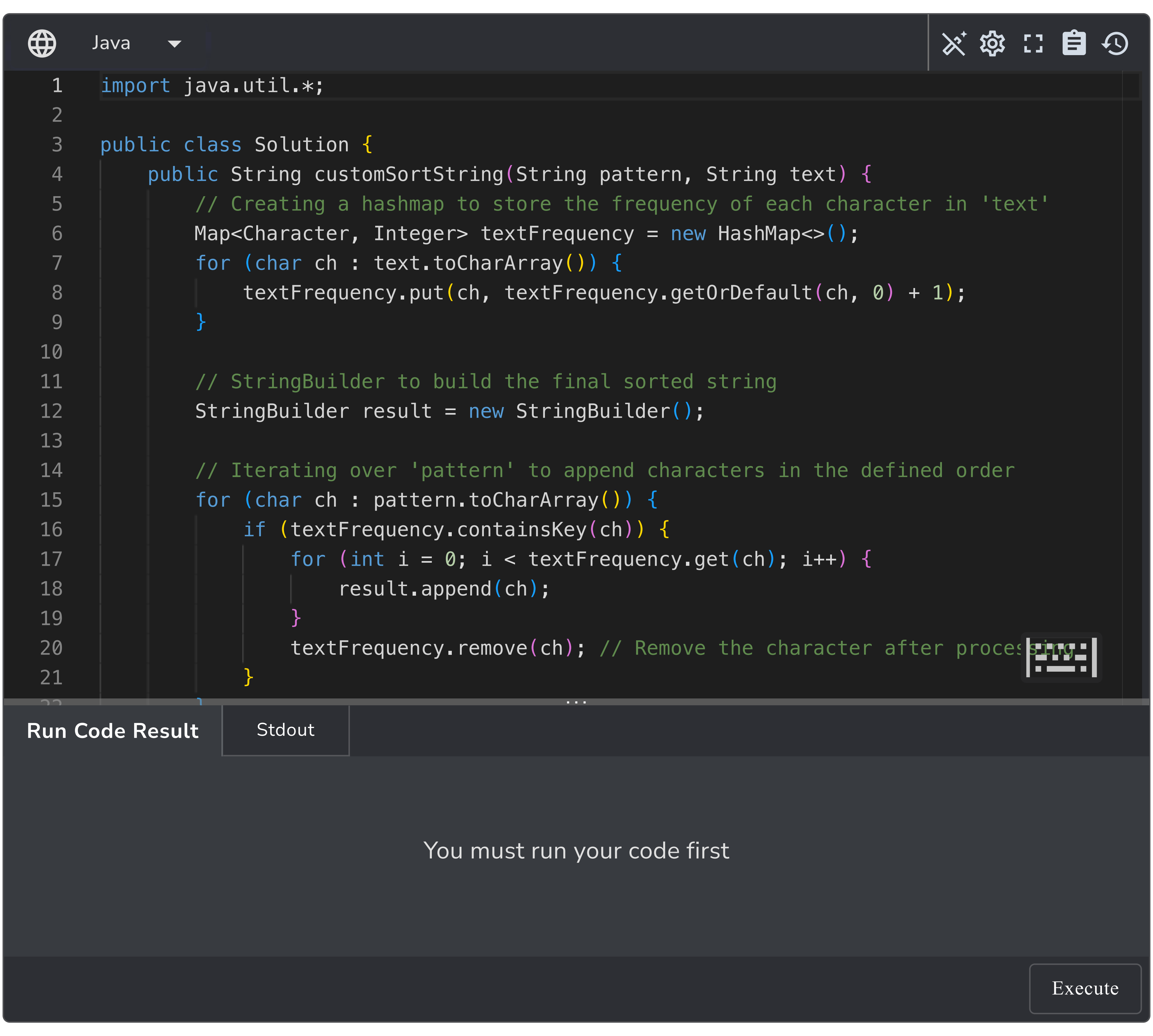
- Create a `HashMap` to store the frequency of each character in 'text'.
- Iterate over 'text' and update the frequency in the hashmap for each character.
- Initialize an empty string `result` for the final sorted string.
- Iterate over `pattern`:
    - For each character in 'pattern', check if it exists in the hashmap.
    - If it exists, append it to `result` as many times as its frequency, and then remove it from the hashmap.
- Iterate over 'text' again:
    - For each character, if it still exists in the hashmap (meaning it was not in 'pattern'), append it to `result`.
- Return `result` as the final output.

Algorithm Walkthrough

Input: `pattern = "mno", text = "onomon"`

1. Create frequency hashmap for 'text': `{o: 3, n: 2, m: 1}`
2. Initialize `result` as an empty string.
3. Iterate over 'pattern':
    - 'm' found in the map, append to `result` -> `result = "m"`
    - 'n' found in the map, append twice -> `result = "mnn"`
    - 'o' found in the map, append thrice -> `result = "mnnooo"`
4. 'm', 'n', and 'o' are removed from hashmap.
5. No additional characters to append from 'text'.
6. Final output is `"mnnooo"`.

Code



```java
import java.util.*;

public class Solution {
    public String customSortString(String pattern, String text) {
        // Creating a hashmap to store the frequency of each character in 'text'
        Map<Character, Integer> textFrequency = new HashMap<>();
        for (char ch : text.toCharArray()) {
            textFrequency.put(ch, textFrequency.getOrDefault(ch, 0) + 1);
        }

        // StringBuilder to build the final sorted string
        StringBuilder result = new StringBuilder();

        // Iterating over 'pattern' to append characters in the defined order
        for (char ch : pattern.toCharArray()) {
            if (textFrequency.containsKey(ch)) {
                for (int i = 0; i < textFrequency.get(ch); i++) {
                    result.append(ch);
                }
                textFrequency.remove(ch); // Remove the character after processing
            }
```

Complexity Analysis

Time Complexity

1. **HashMap Creation**: `O(N)`, where N is the length of 'text'. This is for iterating over 'text' to create the frequency hashmap.
2. **Iterating Over 'Pattern'**: `O(M)`, where M is the length of 'pattern'. This covers iterating over 'pattern' and appending characters to the result.
3. **Final Iteration Over 'Text'**: `O(N)` for appending remaining characters not in 'pattern'.

The overall time complexity is `O(N + M)`, considering both the iterations over 'text' and 'pattern'.

Space Complexity

1. **HashMap**: `O(N)`, where N is the number of distinct characters in 'text'. In the worst case, this could be equal to the length of 'text' if all characters are unique.
2. **Result String**: `O(N)`, equivalent to the length of 'text', as in the worst case, all characters from 'text' are added to the result.

The overall space complexity is `O(N)`, considering the space for the hashmap and the result string.

← Previous
Custom Sort String (Medium)
Next →
Binary Tree Vertical Order Traversal (Medium)
☐ Mark as Completed